WEEK
6

DECORATOR PATTERN



Mr Randell Gaya

Module Originator:
Dr Tom Blanchard,
Dr Lee Clift

TODAY

The Decorator Pattern

- What is it?
- An example implementation.
- The Structure.
- Consequences.
- Adaptations.

DECORATING WHAT?

Intent:

- *"Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality."*

-

***GoF* Motivation**:

- Add responsibilities to individual objects rather than all objects of a class.
- Inheriting responsibilities is static and inflexible.
- Use composition to add responsibility and forward requests to the composed object as necessary.

In plain speak:

- 'Do the extra stuff then forward the call to a composed object.'

DECORATOR PATTERN

An ‘object structure’ pattern.

Lets you **extend** an objects behaviour or compose an objects behaviour at runtime. *(Similar to the Strategy pattern in intent.)*

Decorator vs Strategy:

- *“changing the skin versus changing the guts”* **-GoF**

Uses polymorphism, delegation and a common interface.

Will come across this pattern in the Java Class Library (Java I/O).

```
BufferedInputStream bis =  new BufferedInputStream(new FileInputStream("filename.txt”));
```

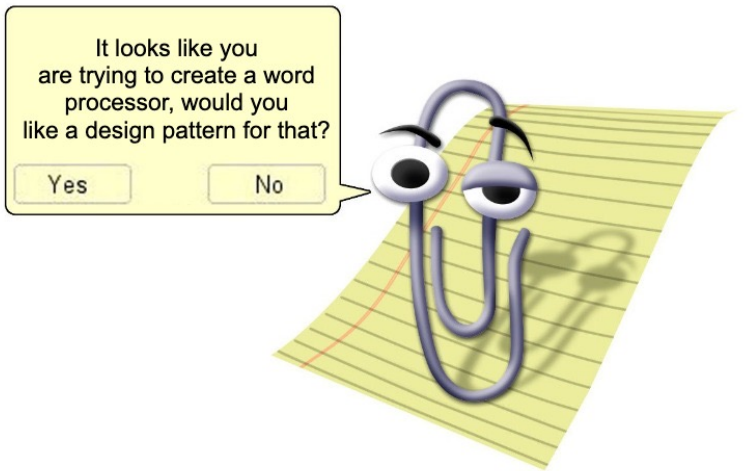
MIKE-ROW-SOFT WORD PROCESSOR

Michael Rowland (or Mike Row to his friends) is a software engineer who creates office software for a living.

Mike needs a software idea to become a billionaire.

“I know! A word processor! You will be able to type on your keyboard, and all the letters will appear on the screen!”

Mike knew this would be a success, and went straight to his desk and began planning his new piece of software.

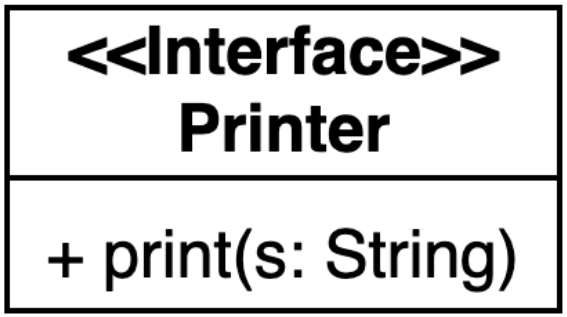


MIKE-ROWE-SOFT WORD PRINCIPLES

Mike's software needs to be able to take strings and write them to the console. *(for the console provides the optimal text viewing experience).*

In the Future we may need to be able to:

- Write strings to a **file** and maybe web sockets (if this www thing ever takes off).
- **Easily modify** the strings before they get written.

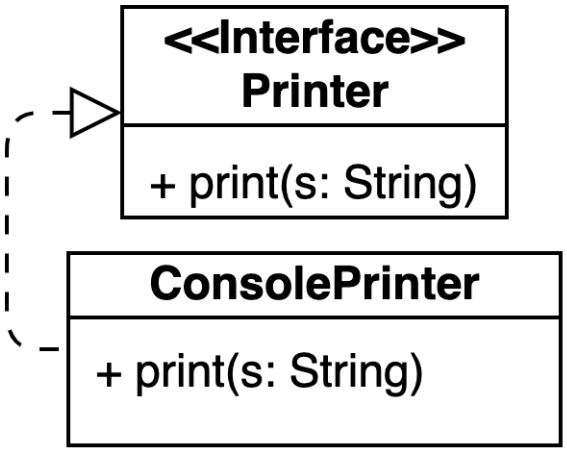



```
public interface Printer {
    public void print(String s);
}
```

Let's use our design principles and first create an interface for things that 'print'.

MIKE-ROW-SOFT WORD PROCESSOR

Mike made his first class that allows him to fill his first requirement; printing to the console.




```
public class ConsolePrinter implements Printer {
    @Override
    public void print(String s){
        System.out.println(s);
    }
}
```

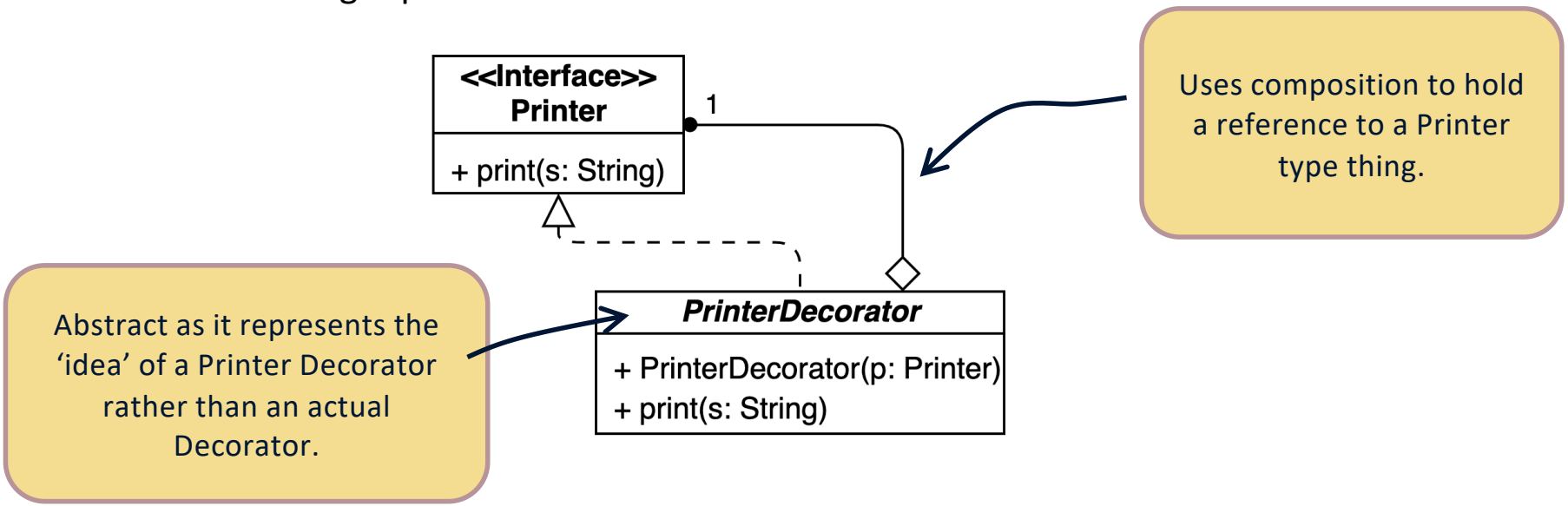
The interface ensures he can easily add different types of Printer in the future without much refactoring.

We can test that it works:

MIKE-ROW-SOFT WORD PROCESSOR

Mike being a **forward-thinking** developer, he wants to implement the **Decorator** pattern ahead of time.

- He knows the markets will change, and may need to provide the ability to change how a string is printed.



He makes a new abstract **PrinterDecorator** class to act as the super class for all other PrinterDecorators.

MIKE-ROW-SOFT WORD EXAMPLE

Let's look at the PrinterDecorator in a bit more detail.

```
public abstract class PrinterDecorator implements Printer{
    private Printer printer;

    public PrinterDecorator(Printer printer){
        this.printer = printer;
    }

    @Override
    public void print(String s) {
        printer.print(s);
    }
}
```

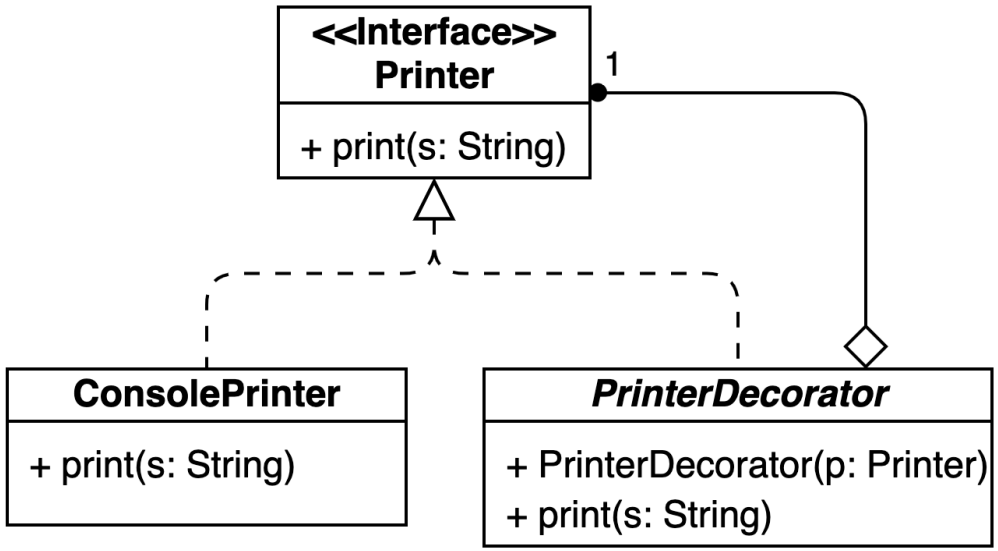
Uses composition to hold a reference to a Printer type thing.

Uses delegation to let the Printer it decorates do the actual printing.

A PrinterDecorator will have a reference to a Printer (to do the actual printing) but will be able to **modify** what is printed.

MIKE-ROW-SOFT WORD EXAMPLE

The current Class Diagram



How might we go about adding an actual Decorator?

Remember, a Decorator changes the behaviour of the object it decorates.

FEATURE CREEP...

Mike gets a new feature request from a user; all strings printed to the console should be able to be made **uppercase**, have **line numbers** added or **both**. *(why is normal printing not enough!?!?)*

- But this is why Mike used the Decorator Pattern!

First, we **encapsulate** those behaviours in their own classes.



They are types of PrinterDecorator so they **extend** the PrinterDecorator abstract class.

MIKE-ROW-SOFT WORD UPPERCASE

Let's take a look at one of these Decorators.

```
public class UpperCasePrinter extends PrinterDecorator{

    public UpperCasePrinter(Printer printer) {
        super(printer);
    }

    @Override
    public void print(String s) {
        super.print(s.toUpperCase());
    }
}
```

This Decorator **is-a** PrinterDecorator

We need to tell which Printer it is Decorating!

We modify the string to be printed (make it uppercase) and then delegate the actual printing to the Printer we were given in the constructor.

Which bit is the decorator and which bit is the decorated object?

```
Printer uppercasePrinter = new UpperCasePrinter(new ConsolePrinter());
printer.print("This is only a test!");
```

MIKE-ROW-SOFT WORD

```
Printer uppercasePrinter = new UpperCasePrinter( new  ConsolePrinter());
uppercasePrinter.print("This is only a test!");

Printer lineNumPrinter = new LineNumberPrinter( new  ConsolePrinter());
lineNumPrinter.print("This is only a test!");
```

Uppercase

Line numbers

Upper case and line number

We can apply uppercase, line numbers or both to our console printing.

```
% java Main
THIS ONLY A TEST!
0: This is only a test!
0: THIS IS ONLY A TEST!
```

MIKE-ROW-SOFT WORD: NOW WITH NUMBERS

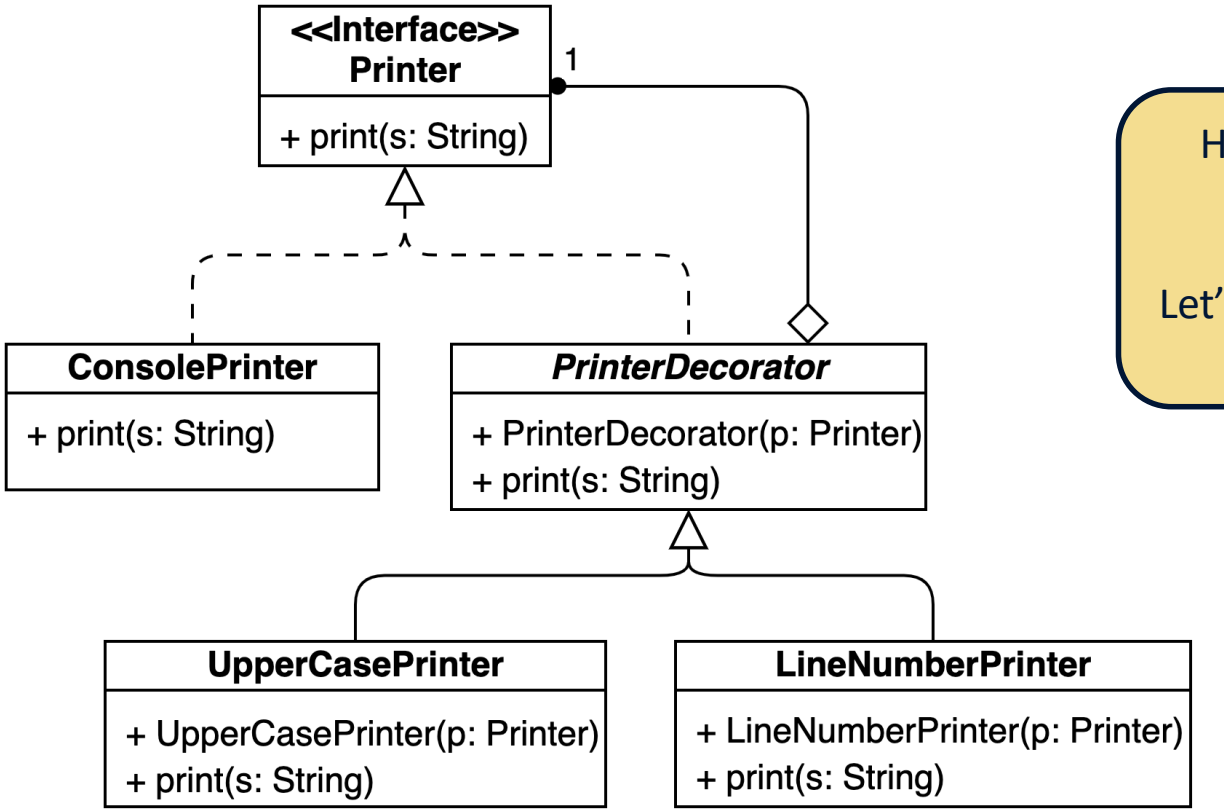
Write the code for the **LineNumberPrinter**.



1. Extend the PrinterDecorator.
2. Provide the extra behaviour.
3. Delegate the printing.

MIKE-ROW-SOFT WORD V3.0

The bigger picture:



How would we **add another Decorator**?

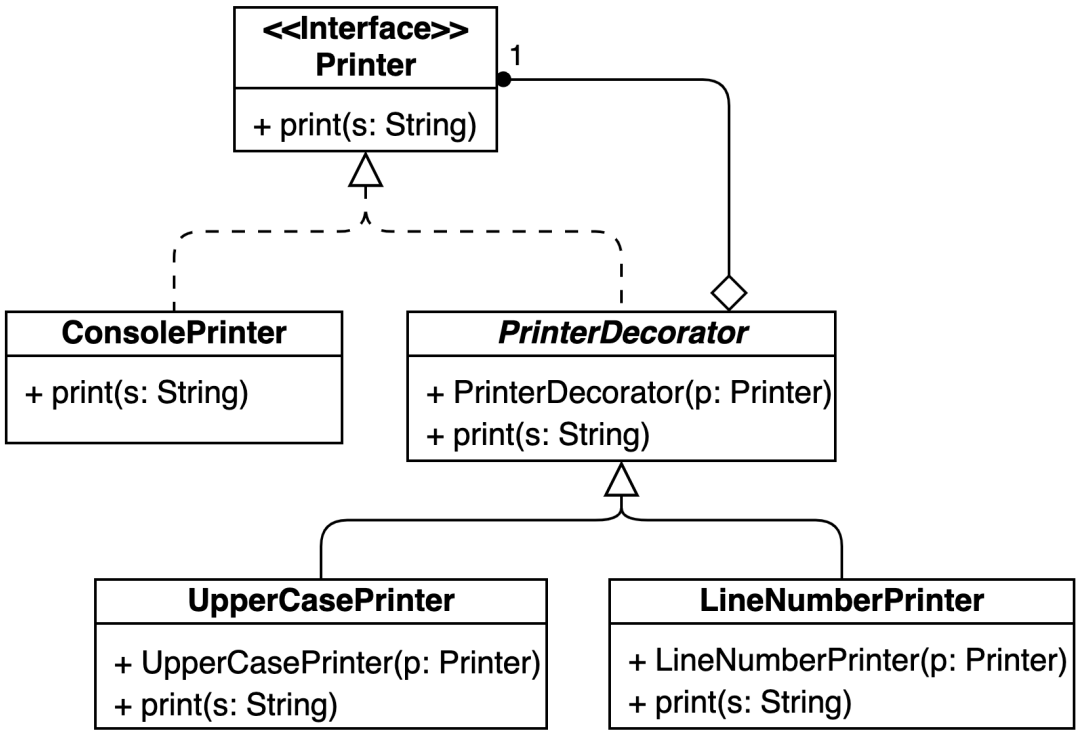
Let's say a we want to be able to wrap long strings?

This is essentially the Class Diagram for the **Decorator Pattern**.
(We will look at the official one in a bit)

MIKE-ROW-SOFT WORD: CONSOLE NO MORE

As mike expected, the time of console printing has passed, and now **writing strings to a file** is all the rage, while retaining the ability to make them **uppercase** and give them **line numbers**.

What **class**(es) and/or **relationship**(s) do we need to add to the Class Diagram?



MIKE-ROW-SOFT WORD

Despite the fact we have added a lot:

- Ability to print to the console.
- Ability to print to a file.
- Make strings uppercase, line numbering, wrapping lines.
- Any combination of the above!

Our client code wouldn't have had to change.

```
public void doComplicatedThingThatNeedsAPrinter(Printer printer){
    //Fancy complicated code that needs to print things
}
```

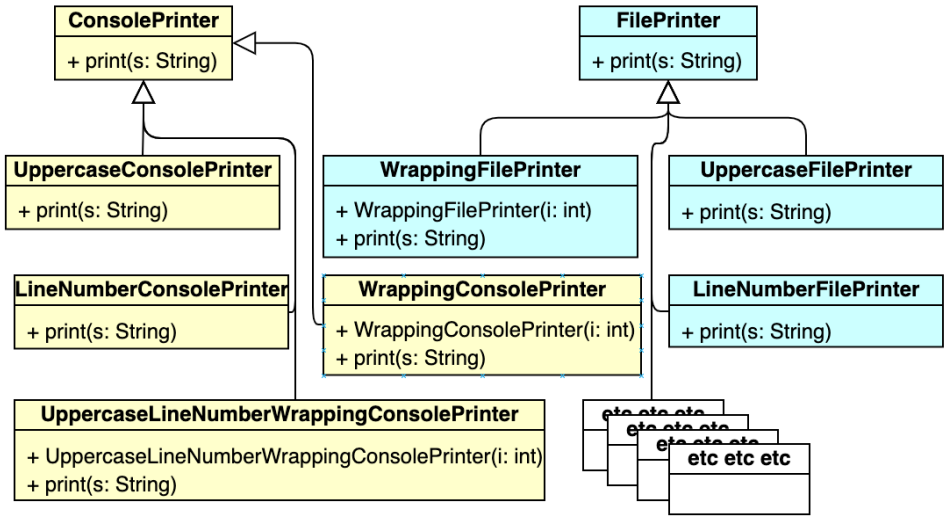
- This code doesn't have to change at all if we used a simple ConsolePrinter or double decorated FilePrinter.
  (I think that is really cool)

No matter what kind of Printer we are using (or decorated printer) the client code doesn't have to change. Adding new behaviour is easy!

A WORSE WAY...

What would have happened if we used **inheritance** for our different behaviours?

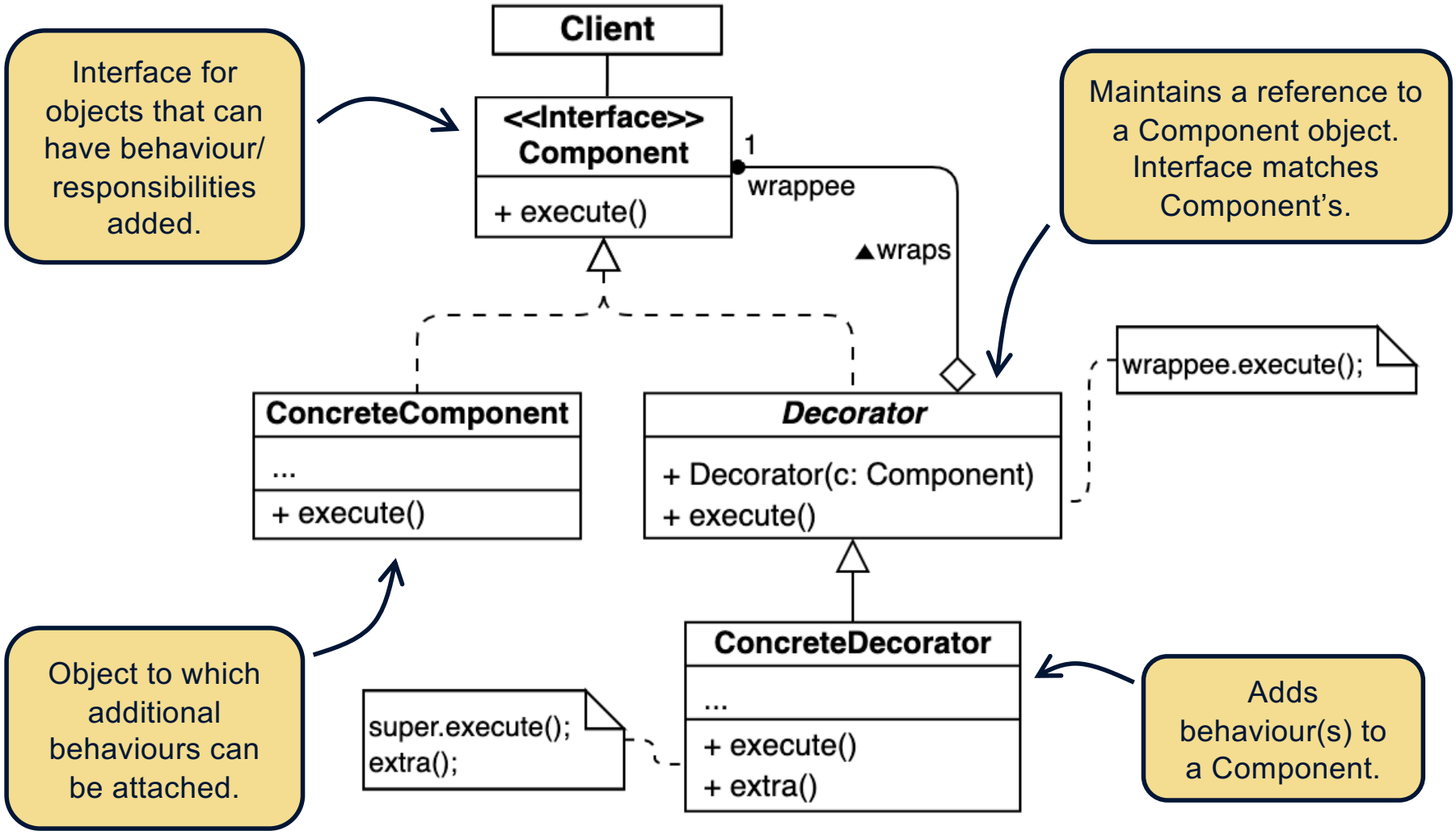
- ConsolePrinter, UppercaseConsolePrinter, LineNumberConsolePrinter, WrappingConsolePrinter, UppercaseLineNumberConsolePrinter, UppercaseWrappingConsolePrinter, LineNumberWrappingConsolePrinter, UppercaseLineNumberWrappingConsolePrinter, FilePrinter, UppercaseFilePrinter… etc etc etc



Adding another type of printer (SocketPrinter) means duplicating the entire inheritance hierarchy again.

Modifying how line wrapping or number works? 🤯🤯🤯🤯

DECORATOR PATTERN STRUCTURE



CONSEQUENCES

Lots of “little objects”.

- Can result in systems composed of many **similar looking** small objects. Easy to use if you understand them but **hard to learn** and debug. Instantiating decorated objects can be… complex.

Offers a **pay-as-you-go** approach to adding behaviours. Instead of trying to support all foreseeable features in one complex class, functionality is added incrementally via Decorators.

A decorator and its component **aren’t identical**.

- If you write code that specifically demands a ConsolePrinter… a decorated ConsolePrinter won’t work!

ADAPTATIONS

Can **drop** the abstract Decorator class if there is **only one responsibility.**

- This then is responsible for forwarding calls to theConcreteDecorator component.

What if a decorator wants to know about other decorators applied to an object?

- Starting to push the intent for this pattern.
- (it is possible)

SUMMARY

Decorator is a really useful pattern to add behaviour to objects dynamically.

Uses:

- Interfaces over Implementations.
- Composition over inheritance.
- Delegation.

Related to Strategy, Composite and Adapter Patterns.

Often used with a Builder and/or Factory Pattern.